

DOXYGEN

Guide pour la documentation C++

Table des matières

1. Philosophie de la documentation Doxygen.....	2
2. En-têtes de fichiers.....	2
2.1. En-tête pour un fichier .h (déclaration de classe).....	2
2.2. En-tête pour un fichier .cpp (implémentation).....	2
2.3. Balises d'en-tête — référence.....	3
3. Documentation des classes.....	3
3.1. Bloc de classe dans le .h.....	3
3.2. Balises utiles pour une classe.....	3
4. Documentation des attributs.....	4
4.1. Commentaire avant l'attribut (style bloc).....	4
4.2. Commentaire après l'attribut (style inline ///<).....	4
5. Documentation des méthodes.....	4
5.1. Constructeur.....	4
5.2. Accesseur (getter).....	5
5.3. Mutateur (setter).....	5
5.4. Opérateur surchargé.....	5
5.5. Référence complète des balises pour une méthode.....	6
6. Exemple complet — classe Compteur.....	6
6.1. Compteur.h.....	6
6.2. Compteur.cpp (extrait).....	7
7. Configuration avec Doxywizard sous Linux Mint.....	8
7.1. Installation (déjà fait sur vos poste de travail).....	8
7.2. Lancement de Doxywizard.....	8
7.3. Paramètres avancés (onglet "Expert").....	10
8. Générer la documentation.....	11
9. Diagrammes UML avec Graphviz.....	12
9.1. Types de diagrammes générés.....	12
9.2. Paramètres Graphviz à ajuster.....	12
9.3. Grouper les classes en modules.....	12
10. Automatisation de la documentation avec GitHub.....	13
10.1. Introduction.....	13
10.2. Principe de fonctionnement.....	13
10.3. Anatomie d'un fichier workflow.....	13
10.4. Autres usages possibles des Actions GitHub.....	14
10.5. Configurer les permissions — le GITHUB_TOKEN.....	14
10.6. Surveiller l'exécution — onglet Actions.....	16
11. Utilisation de la branche gh-pages.....	17
11.1. Pourquoi une branche séparée ?.....	17
11.2. Structure de la branche gh-pages.....	17
11.3. Création automatique.....	17
11.4. Configurer GitHub Pages pour lire gh-pages.....	18
11.5. Cycle de vie complet.....	18
12. Hébergement sur GitHub Pages.....	19
12.1. Structure du dépôt recommandée.....	19
12.2. Préparer le Doxyfile de chaque projet.....	19
12.3. Configurer le .gitignore.....	20
12.4. Créer le dossier .github/workflows/.....	20
12.5. Fichier doxygen.yml — workflow complet.....	20
12.6. Premier déploiement — procédure complète.....	21
12.7. URLs d'accès.....	22
12.8. Récapitulatif des avantages.....	22
13. Checklist — documentation complète.....	22
13.1. Avant de générer.....	22
13.2. Après génération.....	23
13.3. Pour le déploiement GitHub Actions.....	23

1. Philosophie de la documentation Doxygen

Doxygen analyse les commentaires de type `/** ... */` ou `/*! ... */` et génère une documentation HTML, LaTeX ou XML. Pour le C++, la stratégie optimale repose sur une répartition claire entre les fichiers `.h` et `.cpp`. Le tableau ci-dessous résume cette stratégie :

Fichier	Élément	Justification
<code>.h</code>	En-tête de classe	Déclaration : c'est l'interface publique, c'est là que le développeur regarde en premier
<code>.h</code>	Attributs	Liés à la déclaration, proches de la définition de type → le lecteur voit tout d'un coup
<code>.h</code>	Méthodes inline	Définies dans le <code>.h</code> , donc documentées ici
<code>.cpp</code>	Méthodes (corps)	L'implémentation est dans le <code>.cpp</code> → <code>@param</code> , <code>@return</code> , <code>@details</code> ici
<code>.cpp</code>	Fonctions libres	Idem : implémentation dans le <code>.cpp</code>

Règle d'or

Documentez là où se trouve la définition. Pour Doxygen, ce qui compte est que le commentaire `/** */` précède immédiatement la déclaration ou la définition qu'il documente.

Avec `EXTRACT_ALL = YES` dans le Doxyfile, Doxygen fusionne les infos des deux fichiers.

2. En-têtes de fichiers

Chaque fichier source doit commencer par un bloc Doxygen identifiant l'auteur, la date et le rôle du fichier.

2.1. En-tête pour un fichier `.h` (déclaration de classe)

```
/**
 * @file      Temperature.h
 * @author    Prénom NOM <email@domaine.fr>
 * @date      2024-09-15
 * @version   1.0
 * @brief     Déclaration de la classe Temperature.
 *
 *
 * @details   Ce fichier contient l'interface de la classe Temperature,
 *            permettant de stocker et convertir une valeur thermique.
 *            Elle prend en charge les unités Celsius, Fahrenheit et Kelvin.
 */
```

2.2. En-tête pour un fichier `.cpp` (implémentation)

```
/**
 * @file      Temperature.cpp
 * @author    Prénom NOM <email@domaine.fr>
 * @date      2024-09-15
 * @version   1.0
 * @brief     Implémentation de la classe Temperature.
 *
 *
 * @details   Contient le corps de toutes les méthodes déclarées dans
 *            Temperature.h. Voir ce fichier pour la documentation des
 *            attribut de la classe.
 */
```

2.3. Balises d'en-tête — référence

Balise	Exemple	Rôle
@file	Temperature.h	Nom du fichier (obligatoire pour Doxygen)
@author	Prénom NOM <email>	Auteur(s) du fichier
@date	2024-09-15	Date de création (format ISO recommandé)
@version	1.0	Version du fichier
@brief	Description courte	Résumé affiché dans les listes Doxygen
@details	Description longue	Détails affichés dans la page dédiée
@copyright	MIT License	Licence applicable

3. Documentation

3.1. Bloc de classe dans le .h

Le commentaire de classe se place immédiatement avant le mot-clé `class`.

```
/**
 * @class Temperature
 * @brief Représente une température avec conversion d'unités.
 *
 * @details La classe Temperature stocke une valeur en degrés Celsius
 *          en interne et fournit des accesseurs pour les conversions.
 *          Elle lève une exception std::invalid_argument si la valeur
 *          descend sous le zéro absolu (-273.15 °C).
 *
 * @note Toutes les valeurs sont stockées en double précision.
 *
 * @see Compteur, IPv4
 *
 * Exemple d'utilisation :
 * @code
 * Temperature t(20.0);
 * std::cout << t.EnFahrenheit() << std::endl; // 68.0
 * @endcode
 */
class Temperature
{
    // ...
};
```

3.2. Balises utiles pour une classe

Balise	Rôle
@class	Nom de la classe (utile si le commentaire est hors du fichier)
@brief	Description d'une ligne (résumé dans les index)
@details	Description complète (peut s'étendre sur plusieurs paragraphes)
@note	Note importante pour l'utilisateur
@warning	Avertissement (ex. : non thread-safe)
@todo	Fonctionnalité à implémenter
@see	Renvoi vers d'autres classes ou méthodes liées
@code / @endcode	Extrait de code dans la documentation

4. Documentation des attributs

Les attributs sont documentés dans le fichier `.h`, directement avant ou après la déclaration.

4.1. Commentaire avant l'attribut (style bloc)

```
private:
    /** @brief Valeur de la température en degrés Celsius. */
    double valeur;

    /**
     * @brief Unité d'affichage par défaut.
     * @details Peut valoir "C", "F" ou "K".
     *          Modifiable via SetUnite().
     */
    std::string unite;
```

4.2. Commentaire après l'attribut (style inline ///)

La syntaxe `///
<` documente l'élément qui précède sur la même ligne ou la ligne précédente.

```
private:
    double valeur;          ///  
    std::string unite;      ///  
    bool estValide;         ///  
absolu.                    ///  
                           true si la valeur est au-dessus du zéro
```

Conseil

Préférez le style `///
<` pour les attributs simples (une seule ligne).
Utilisez le style bloc `/** */` dès qu'une explication `@details` est nécessaire.
Ne doublez pas : si `///
<` est suffisant, n'ajoutez pas de bloc `/** */`.

5. Documentation des méthodes

Le corps des méthodes est dans le `.cpp` : c'est donc là que se place le bloc Doxygen complet.

5.1. Constructeur

```
/**
 * @brief Construit un objet Temperature avec une valeur initiale.
 *
 * @param _valeur  Température initiale en degrés Celsius.
 * @param _unite   Unité d'affichage : "C", "F" ou "K" (défaut "C").
 *
 * @throws std::invalid_argument Si _valeur < -273.15.
 */
Temperature::Temperature(double _valeur, const std::string& _unite)
    : valeur(_valeur), unite(_unite)
{
    if (valeur < -273.15)
        throw std::invalid_argument("Température sous le zéro absolu.");
}
```

5.2. Accesseur (getter)

```
/**
 * @brief   Retourne la température convertie en degrés Fahrenheit.
 *
 * @details Formule :  $F = (C \times 9/5) + 32$ .
 *
 * @return  double  Valeur en Fahrenheit.
 */
double Temperature::EnFahrenheit() const
{
    return (valeur * 9.0 / 5.0) + 32.0;
}
```

5.3. Mutateur (setter)

```
/**
 * @brief   Modifie la valeur de la température.
 *
 * @param   _valeur  Nouvelle valeur en degrés Celsius.
 *
 * @throws  std::invalid_argument Si _valeur < -273.15.
 *
 * @note    L'unité d'affichage n'est pas modifiée par cette méthode.
 */
void Temperature::SetValeur(double _valeur)
{
    if (_valeur < -273.15)
        throw std::invalid_argument("Température sous le zéro absolu.");
    valeur = _valeur;
}
```

5.4. Opérateur surchargé

```
/**
 * @brief   Opérateur d'addition entre deux températures.
 *
 * @param   _other  Température à additionner.
 *
 * @return  Temperature  Nouvelle instance représentant la somme.
 *
 * @warning Le résultat peut dépasser les limites physiques.
 */
Temperature Temperature::operator+(const Temperature& _other) const
{
    return Temperature(valeur + _other.valeur, unite);
}
```

5.5. Référence complète des balises pour une méthode

Balise	Utilisation
@brief	Description courte, obligatoire
@details	Explication détaillée de l'algorithme ou du comportement
@param _nom	Documente un paramètre (préfixe _ respecté)
@param[in] _nom	Paramètre en entrée uniquement (lecture)
@param[out] _nom	Paramètre en sortie uniquement (résultat)
@param[in,out] _nom	Paramètre en entrée et en sortie
@return	Valeur de retour et son interprétation
@throws / @exception	Exception levée et condition de déclenchement
@pre	Précondition (état attendu avant l'appel)
@post	Postcondition (état garanti après l'appel)
@note	Note d'implémentation
@warning	Point d'attention critique
@see	Lien vers une autre méthode ou classe

6. Exemple complet — classe Compteur

6.1. Compteur.h

```

/**
 * @file      Compteur.h
 * @author    Prénom NOM <email@domaine.fr>
 * @date      2024-09-15
 * @brief     Déclaration de la classe Compteur.
 * @details   Compteur incrémentable et décrémentable avec valeur min/max.
 */
#ifndef COMPTEUR_H
#define COMPTEUR_H
/**
 * @class Compteur
 * @brief Compteur entier borné avec saturation.
 *
 * @details Le compteur ne dépasse jamais valeurMax ni ne descend
 *          sous valeurMin. Les opérations hors-bornes saturent.
 */
class Compteur
{
public:
    Compteur(int _min, int _max, int _initial = 0);
    ~Compteur();
    void    Incrémenter();
    void    Décrémenter();
    int     GetValeur() const;
    bool     EstAuMax() const;
    bool     EstAuMin() const;
private:
    int valeur;           ///< Valeur courante du compteur.
    int valeurMin;        ///< Borne inférieure (inclusive).
    int valeurMax;        ///< Borne supérieure (inclusive).
};
#endif // COMPTEUR_H

```

6.2. Compteur.cpp (extrait)

```
/**
 * @file      Compteur.cpp
 * @author    Prénom NOM <email@domaine.fr>
 * @date      2024-09-15
 * @brief     Implémentation de la classe Compteur.
 */
#include "Compteur.h"

/**
 * @brief     Construit un Compteur avec ses bornes et sa valeur initiale.
 *
 * @param     _min      Valeur minimale autorisée.
 * @param     _max      Valeur maximale autorisée.
 * @param     _initial  Valeur de départ (doit être entre _min et _max).
 *
 * @pre       _min <= _initial <= _max
 */
Compteur::Compteur(int _min, int _max, int _initial)
    : valeur(_initial)
    , valeurMin(_min)
    , valeurMax(_max)
{
}

/**
 * @brief     Incrémente le compteur d'une unité.
 *
 * @details   Si la valeur courante est déjà égale à valeurMax,
 *           l'appel est sans effet (saturation haute).
 *
 * @post      valeur <= valeurMax
 */
void Compteur::Incrementer()
{
    if (valeur < valeurMax)
        valeur++;
}

/**
 * @brief     Retourne la valeur courante du compteur.
 *
 * @return    int  Valeur courante.
 */
int Compteur::GetValeur() const
{
    return valeur;
}
```

7. Configuration avec Doxywizard sous Linux Mint

7.1. Installation (déjà fait sur vos poste de travail)

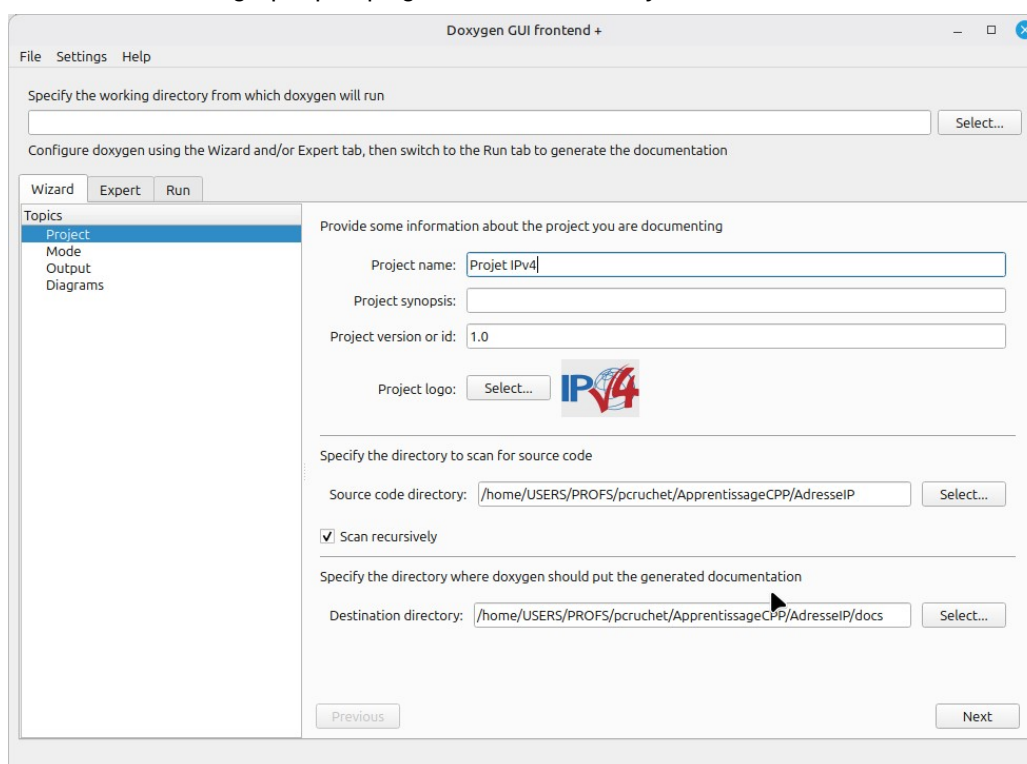
```
# Installer Doxygen + Doxywizard + Graphviz (diagrammes UML)
sudo apt update
sudo apt install doxygen doxygen-gui graphviz

# Vérifier les versions installées
doxygen --version
dot -V
```

7.2. Lancement de Doxywizard

```
doxywizard &
```

Doxywizard est une interface graphique qui génère le fichier Doxyfile.

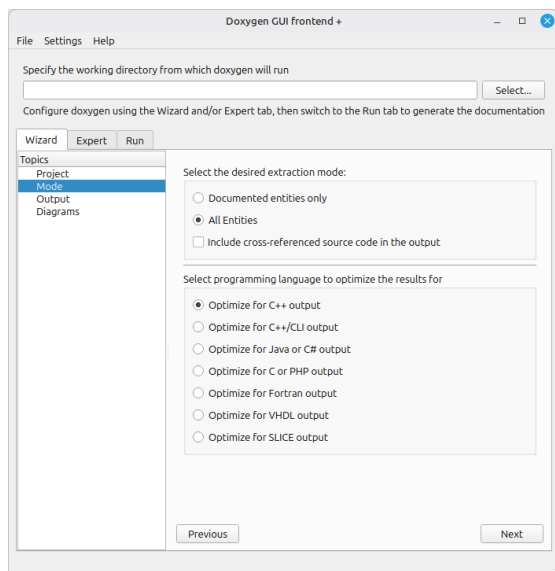


Étape 1 — Onglet "Wizard > Project"

- Nom du projet : Cours C++ BTS CIEL
- Version : 1.0
- Répertoire source : /home/user/mon_projet/src
- Répertoire sortie : /home/user/mon_projet/docs
- ☒ Analyser les sous-répertoires récursivement

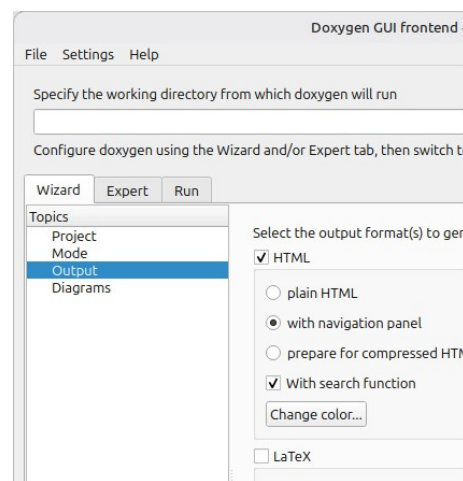
Attention aux chemins dans Doxywizard

Doxywizard remplit souvent les champs avec le chemin absolu de votre machine :
 /home/USERS/PROFS/pcruchet/ApprentissageCPP/AdresselP/
 Ce chemin n'existe pas sur le runner GitHub → le workflow échouera.
 Utilisez toujours "." pour la source et "docs" pour la sortie,
 ou corrigez ensuite avec : `sed -i 's|^INPUT\s*=\s*.*|INPUT = .|' Doxyfile`



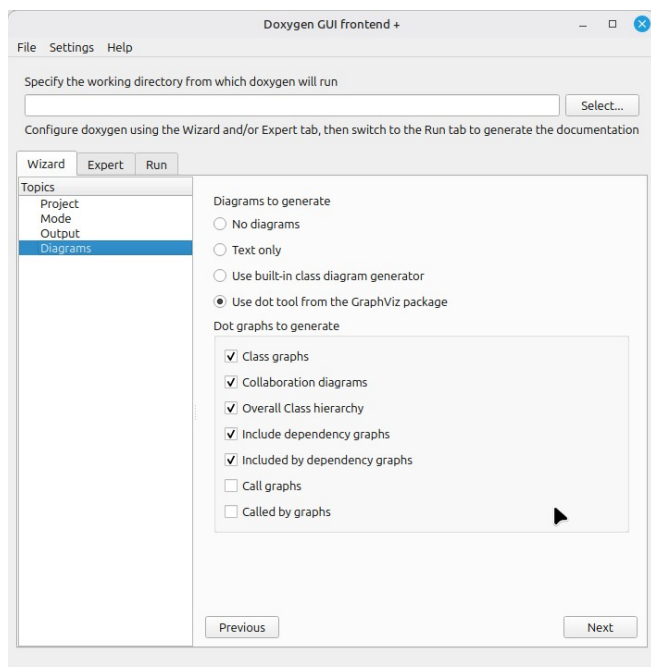
Étape 2 — Onglet "Wizard > Mode"

- Mode : All Entities (toutes les entités, même non documentées)
- Language : Optimized for C++ output



Étape 3 — Onglet "Wizard > Output"

- ☒ HTML → with navigation panel → with search function
- ☐ LaTeX (décocher si HTML seul suffit)



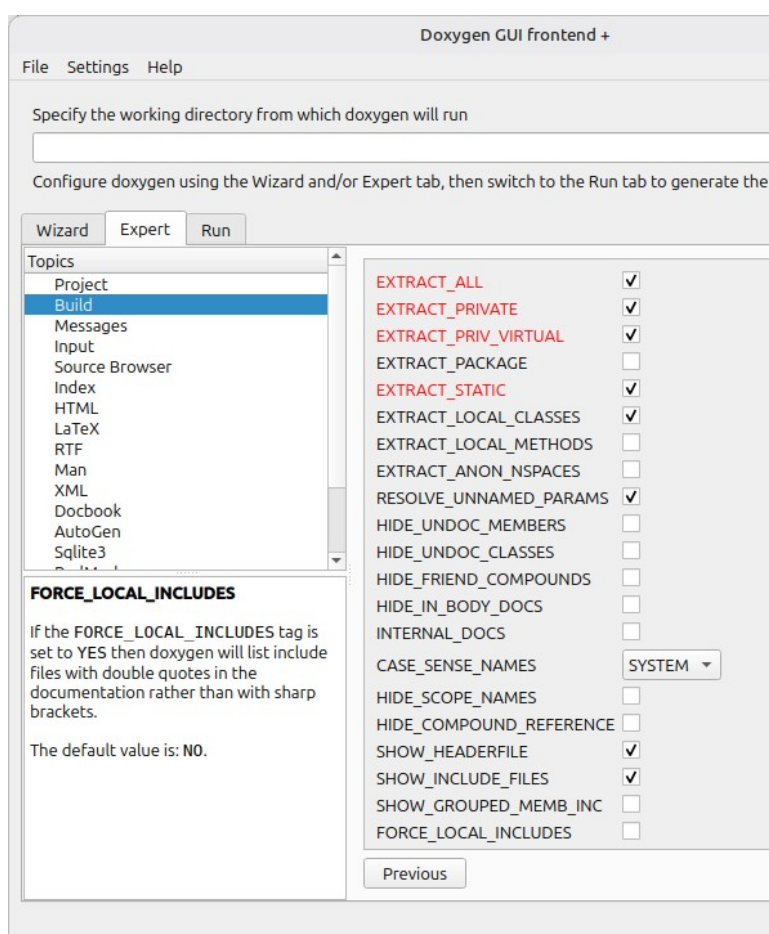
Étape 4 — Onglet "Wizard > Diagrams"

- ☒ Use dot tool from the GraphViz package
- ☒ Class diagrams
- ☒ Collaboration diagrams
- ☒ Include dependency graphs
- ☒ Included by dependency graphs
- ☒ Call graphs
- ☒ Called-by graphs

7.3. Paramètres avancés (onglet "Expert")

Ces réglages sont accessibles via l'onglet Expert ou en éditant directement le Doxyfile. Les paramètres en **gras** sont ceux à vérifier en priorité :

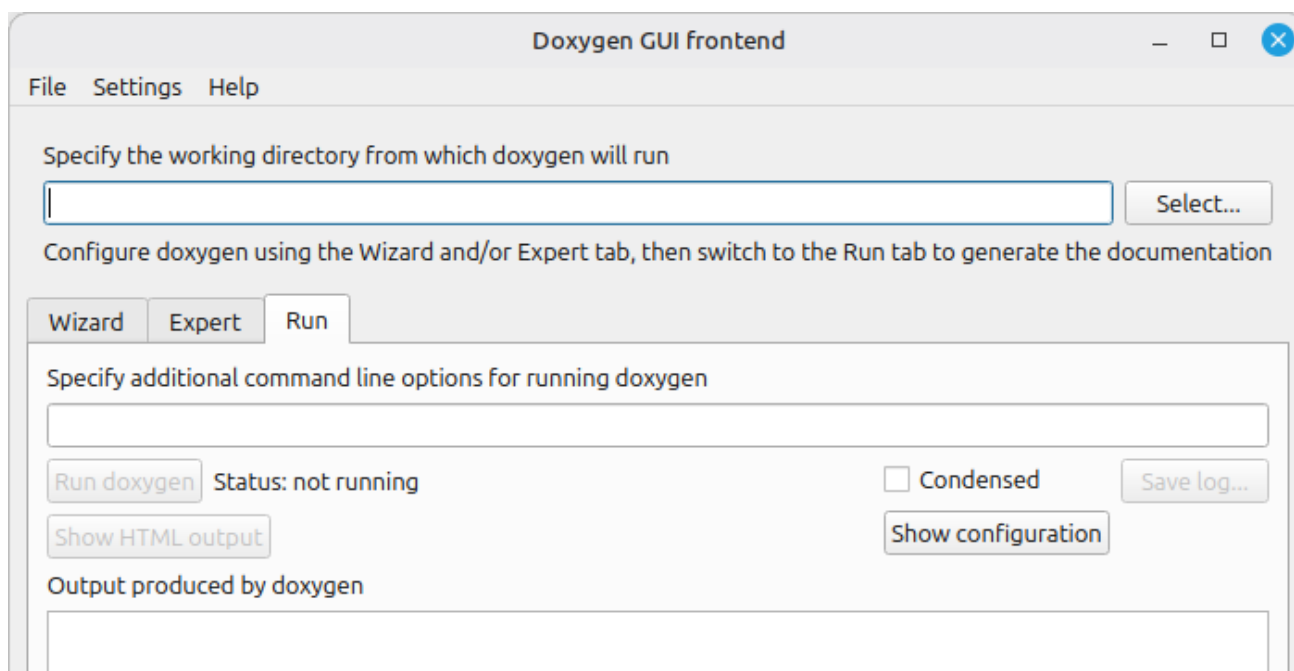
Paramètre Doxyfile	Valeur recommandée	Effet
EXTRACT_ALL	YES	Documente même les entités sans commentaire
EXTRACT_PRIVATE	YES	Inclut les membres privés
EXTRACT_STATIC	YES	Inclut les membres statiques
HAVE_DOT	YES	Active Graphviz pour les diagrammes
UML_LOOK	YES	Diagrammes au style UML (boîtes structurées)
CALL_GRAPH	YES	Graphe d'appels pour chaque méthode
CALLER_GRAPH	YES	Graphe des appelants
GENERATE_TREEVIEW	YES	Panneau de navigation latéral
HTML_DYNAMIC_SECTIONS	YES	Sections repliables dans le HTML
SEARCHENGINE	YES	Barre de recherche JavaScript
SOURCE_BROWSER	YES	Affiche le code source dans la doc
INLINE_SOURCES	NO	Code source en dehors des pages de méthodes
GENERATE_LATEX	NO	Désactive la sortie LaTeX (HTML seul)
OUTPUT_LANGUAGE	French	Interface Doxygen en français



En **rouge** les paramètres qui ont été modifiés.

En fonction des réglages souhaités, il est parfois nécessaires de configurer les différents Topics.

8. Générer la documentation



```
# Méthode 1 : depuis Doxywizard → bouton "Run doxygen"

# Méthode 2 : en ligne de commande
cd /home/user/mon_projet
doxygen Doxyfile

# Ouvrir la documentation générée
xdg-open docs/html/index.html
```

Conseil

Enregistrez toujours la configuration dans le fichier Doxyfile (bouton "**Save**" dans Doxywizard). Cela permet de régénérer en ligne de commande et est indispensable pour le workflow GitHub Actions.

9. Diagrammes UML avec Graphviz

Avec HAVE_DOT = YES et UML_LOOK = YES, Doxygen génère automatiquement des diagrammes de type UML pour chaque classe.

9.1. Types de diagrammes générés

Type de diagramme	Ce qu'il montre
Diagramme d'héritage	Hiérarchie de classes (parent → enfant)
Diagramme de collaboration	Relations entre objets (composition, agrégation)
Graphe d'inclusion	Dépendances entre fichiers .h
Graphe d'appels (Call graph)	Méthodes appelées par une méthode donnée
Graphe des appelants	Qui appelle une méthode donnée

9.2. Paramètres Graphviz à ajuster

```
# Dans le Doxyfile ou onglet Expert > Dot
DOT_IMAGE_FORMAT    = svg      # Meilleure qualité que PNG
DOT_GRAPH_MAX_NODES = 50      # Limite de nœuds par diagramme
MAX_DOT_GRAPH_DEPTH = 3      # Profondeur maximale d'héritage
INTERACTIVE_SVG     = YES     # Diagrammes SVG cliquables
```

Astuce UML

Utilisez @relates et @memberof pour créer des relations sémantiques entre classes.

La balise @ingroup permet de regrouper les classes dans des modules logiques

(déclaration du groupe avec @defgroup, puis @addtogroup dans chaque fichier).

9.3. Grouper les classes en modules

La balise @defgroup permet de regrouper les classes dans des modules logiques, visibles comme des onglets dans la documentation HTML :

```
/**
 * @defgroup Capteurs Classes de gestion des capteurs
 * @{
 */

/**
 * @ingroup Capteurs
 * @class Temperature
 * @brief Gestion d'une valeur thermique.
 */
class Temperature { /* ... */ };

/** @} */
```

10. Automatisation de la documentation avec GitHub

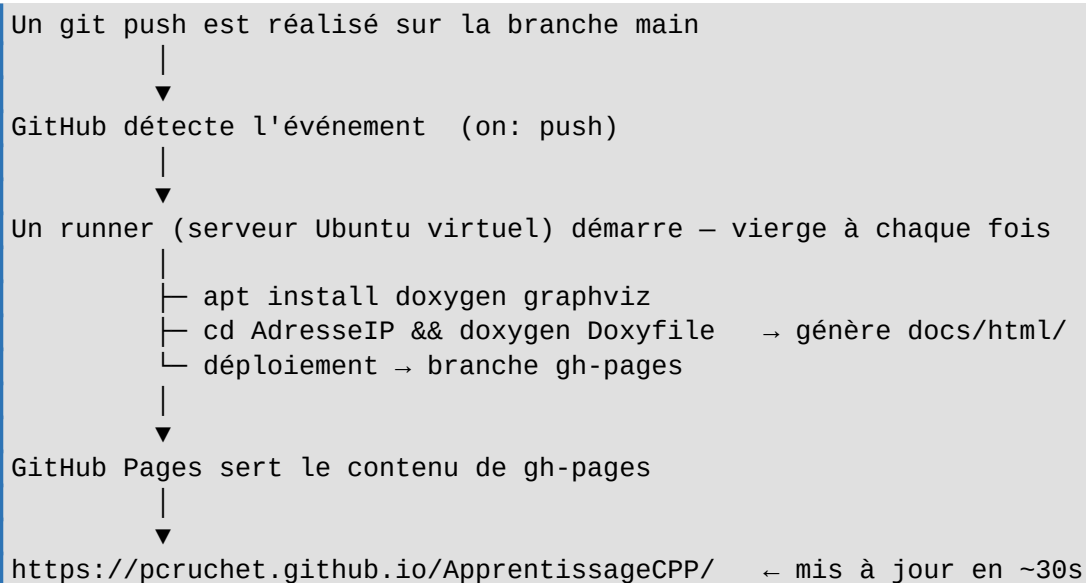
10.1. Introduction

GitHub Actions est le système d'**intégration et déploiement continu (CI/CD)** intégré à GitHub. Il permet d'automatiser des tâches à chaque fois qu'un événement se produit sur le dépôt : un push, une pull request, un déclenchement manuel, etc.

Dans le cadre du cours C++ et de la gestion de projet, on peut l'utiliser pour **régénérer et publier automatiquement** la documentation Doxygen à chaque push, sans aucune intervention manuelle.

10.2. Principe de fonctionnement

Tout repose sur un fichier YAML placé dans `.github/workflows/`. GitHub le lit et exécute les instructions sur un serveur distant appelé **runner**.



10.3. Anatomie d'un fichier workflow

Un fichier `.yml` est composé de blocs structurés. Voici les éléments clés :

Bloc YAML	Rôle	Exemple
name	Nom affiché dans l'onglet Actions	Documentation Doxygen
on	Événements déclencheurs	push, workflow_dispatch
on.push.branches	Branches concernées	[main]
jobs	Liste des tâches à exécuter	deploy-AdresseIP:
runs-on	Système du runner	ubuntu-latest
permissions	Droits accordés au workflow	contents: write
steps	Étapes séquentielles du job	- name: Installer...
uses	Action GitHub réutilisable	actions/checkout@v4
run	Commande shell à exécuter	doxygen Doxyfile
working-directory	Dossier de travail pour la commande	AdresseIP
with	Paramètres d'une action réutilisable	branch: gh-pages

10.4. Autres usages possibles des Actions GitHub

GitHub Actions peut automatiser bien d'autres tâches que la documentation :

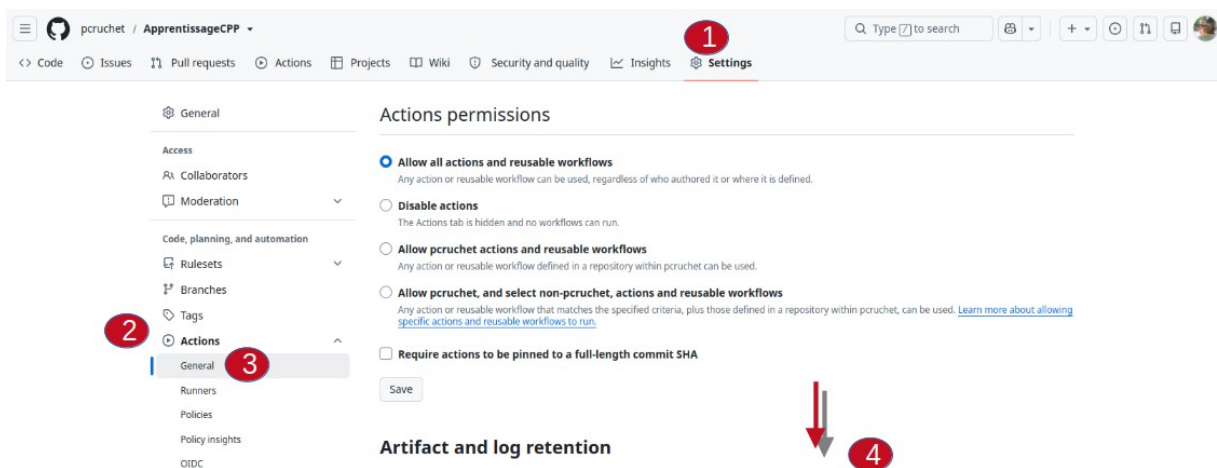
- Compilation et tests automatiques à chaque push (CI)
- Déploiement d'une application web ou d'une API
- Vérification du style de code (linting, clang-format)
- Génération de releases et de fichiers binaires
- Envoi de notifications (email, Slack) en cas d'échec
- Mise à jour automatique de badges de statut dans le README

10.5. Configurer les permissions — le GITHUB_TOKEN

Pour que le workflow puisse écrire sur la branche gh-pages, il doit avoir le droit d'écriture sur le dépôt. Ce droit est accordé via le **GITHUB_TOKEN** — un jeton temporaire généré automatiquement par GitHub pour chaque exécution.

Activer la permission pour le workflow (une seule fois par dépôt)

- GitHub.com → votre dépôt ApprentissageCPP
- → Settings (onglet en haut)
- → Actions (menu gauche)
- → General (sous-menu)
- → Faire défiler jusqu'à "Workflow permissions"



Workflow permissions

Choose the default permissions granted to the GITHUB_TOKEN when running workflows in this repository. You can specify more granular permissions in the workflow using YAML. [Learn more about managing permissions.](#)

- 5
- ☒ **Read and write permissions**
Workflows have read and write permissions in the repository for all scopes.
 - ☐ **Read repository contents and packages permissions**
Workflows have read permissions in the repository for the contents and packages scopes only.

- ☒ Read and write permissions ← cocher cette option
- ☐ Read-only permissions

- → Cliquer Save

Symptôme si la permission manque

Le workflow s'exécute mais échoue à l'étape "Déployer sur GitHub Pages" :
 refusing to allow a Personal Access Token to create or update workflow
 Error: Action failed with "not found"

Solution : activer "Read and write permissions" dans Settings → Actions → General.

Alternative via le Personal Access Token (PAT)

Si vous préférez utiliser un PAT personnel, il doit avoir le scope "workflow" activé :

- GitHub.com → votre profile (icône en haut à droite)
- → Settings → Developer settings
- → Personal access tokens → Tokens (classic) → sélectionner le Token
- → Cocher la case ☒ workflow → Update token

Edit personal access token (classic)

If you've lost or forgotten this token, you can regenerate it, but be aware that any scripts or applications using this token will need to be updated.

[Regenerate token](#)

Note

Token Github ***

What's this token for?

Expiration

This token has no expiration date. To set a new expiration date, you must [regenerate the token](#).

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☒ workflow	Update GitHub Action workflows

Le GITHUB_TOKEN automatique est **plus sûr qu'un PAT** (Personal Access Token) : il expire après chaque run, est limité au seul dépôt concerné et ne nécessite aucune configuration manuelle. Un PAT personnel n'est utile que pour des workflows inter-dépôts.

Bloc permissions dans le workflow

jobs:

 deploy-AdresseIP:

 permissions:

 contents: write ← autorise l'écriture sur gh-pages

Ce bloc est obligatoire dans chaque job qui déploie.

Sans lui, le GITHUB_TOKEN n'a que les droits en lecture.

10.6. Surveiller l'exécution — onglet Actions

Chaque exécution est visible dans **GitHub** → **ApprentissageCPP** → **onglet Actions**. C'est l'endroit où diagnostiquer les problèmes.

Lire la liste des runs

Icône	Signification	Action à faire
 Jaune	Workflow en cours d'exécution	Attendre (~30 secondes)
 Vert	Succès — documentation déployée	Vérifier l'URL GitHub Pages
 Rouge	Échec — une étape a planté	Cliquer pour lire les logs
 Gris	Ignoré (filtre paths non déclenché)	Vérifier le déclencheur on:

Lire les logs d'un run échoué

- Cliquer sur le run rouge dans la liste
- Cliquer sur le job "deploy-docs"
- Chaque étape affiche  ou  avec sa durée
- Cliquer sur l'étape  pour dérouler les logs
- Lire les dernières lignes : l'erreur est toujours à la fin

Déclencher manuellement sans faire de commit

Si `workflow_dispatch:` est présent dans le bloc `on:`, vous pouvez lancer le workflow sans commit :

- GitHub → Actions → "Documentation Doxygen" (liste à gauche)
- → bouton "Run workflow" (à droite)
- → Branch : main → "Run workflow"

Récapitulatif GitHub Actions

- jFichier YAML dans `.github/workflows/` → lu et exécuté par GitHub
- Runner = serveur Ubuntu vierge redémarré pour chaque run
- GITHUB_TOKEN = jeton automatique, suffisant pour gh-pages
- Permissions : Settings → Actions → General → Read and write permissions
- Surveillance : onglet Actions → run → job → étape en erreur
- Déclenchement manuel : bouton "Run workflow" dans l'onglet Actions

11. Utilisation de la branche gh-pages

La branche **gh-pages** est une branche Git ordinaire, mais avec un rôle particulier : GitHub Pages la surveille et sert son contenu comme un site web statique. Elle est **entièrement gérée par le workflow** — vous n'y touchez jamais directement.

11.1. Pourquoi une branche séparée ?

Mettre la documentation dans **main** serait plus simple, mais pose des problèmes :

Aspect	Dans main (déconseillé)	Dans gh-pages (recommandé)
Contenu	Code source + HTML générés mélangés	HTML généré uniquement
git log	Pollué par des commits "docs: régénération"	Propre — seul le code source
Taille dépôt	Grossit vite (SVG, HTML, JS)	Isolé — n'alourdit pas main
Collaboration	Les PR mixent code et doc	Seul le code est reviewé
Clarté	On ne sait plus ce qui est source ou généré	Séparation claire

11.2. Structure de la branche gh-pages

Après le premier run réussi, la branche gh-pages contient uniquement les fichiers HTML/CSS/JS, organisés par projet grâce au paramètre target-folder :

```
gh-pages (branche – visible sur GitHub)
├── AdresseIP/                ← target-folder: AdresseIP
│   ├── index.html
│   ├── classIPv4.html
│   ├── .nojekyll
│   └── ...
├── Compteur2/                ← target-folder: Compteur2
│   ├── index.html
│   └── ...
└── FormeGeometrique/
    ├── index.html
    └── ...
```

Les paramètres target-folder et clean: false

target-folder: AdresseIP → le job écrit uniquement dans le sous-dossier AdresseIP/
les autres projets ne sont pas touchés.

clean: false → ne pas effacer le contenu existant de gh-pages.
Sans cette option, déployer AdresseIP/ effacerait Compteur2/.

11.3. Création automatique

La branche gh-pages **n'existe pas au départ — c'est normal**. Elle est créée automatiquement par l'action lors du premier run réussi. Ne la créez jamais manuellement.

```
# Ne jamais faire :
git checkout -b gh-pages    ← inutile, le workflow le fait

# Ce que fait l'action automatiquement :
1. Vérifie si gh-pages existe
2. Si non → la crée (commit initial vide)
3. Copie le contenu de folder: dans target-folder:
4. git commit + git push sur gh-pages
5. GitHub Pages détecte le push → met le site à jour (~30s)
```

11.4. Configurer GitHub Pages pour lire gh-pages

Configuration à faire **une seule fois** dans Settings → Pages, **après** le premier run réussi :

```
GitHub.com → ApprentissageCPP → Settings → Pages

Source : Deploy from a branch
Branch : gh-pages ▼ / (root) ▼
        (la branche apparaît après le 1er run réussi)
→ Save

# URL d'accès générée :
https://pcruchet.github.io/ApprentissageCPP/
```

Ne pas configurer Pages sur main / /docs

Si Pages pointe sur "main / /docs", GitHub lance Jekyll sur vos sources et échoue avec : "No such file or directory @ dir_chdir0 - /github/workspace/docs". La source doit être "GitHub Actions" ou "gh-pages / (root)".

gh-pages n'apparaît pas dans le menu Branch ?

C'est normal si le workflow n'a pas encore tourné avec succès. Vérifiez que le workflow est vert (✓) dans l'onglet Actions, puis rechargez la page Settings → Pages. La branche gh-pages n'existe que si au moins un run s'est terminé en ✓.

11.5. Cycle de vie complet

```
git push (fichier .h ou .cpp modifié)
|
▼ GitHub Actions se déclenche
Runner Ubuntu :
1. checkout main
2. apt install doxygen graphviz
3. cd AdresseIP && doxygen Doxyfile
4. touch AdresseIP/docs/html/.nojekyll

JamesIves deploy-action :
5. checkout gh-pages
6. copie docs/html/ → gh-pages/AdresseIP/
7. git commit + git push sur gh-pages

GitHub Pages détecte le push sur gh-pages
https://pcruchet.github.io/ApprentissageCPP/AdresseIP/
← mise à jour visible en ~30 secondes
```

Résumé gh-pages

- gh-pages = branche Git dont le contenu est servi comme site web
- Créée automatiquement au 1er run réussi — jamais à la main
- Ne contient que le HTML généré — jamais le code source C++
- target-folder isole chaque projet dans son propre sous-dossier
- clean: false protège les autres projets lors du déploiement
- Configuration GitHub Pages : une seule fois dans Settings → Pages

12. Hébergement sur GitHub Pages

La section 11 a décrit le rôle et le fonctionnement de la branche gh-pages. Cette section détaille la mise en place concrète pour gérer plusieurs projets C++ dans le même dépôt.

12.1. Structure du dépôt recommandée

Le dépôt **ApprentissageCPP** contient un sous-répertoire par projet. Chaque projet possède son propre Doxyfile et son dossier docs/ généré localement mais **non versionné** dans Git.

```
ApprentissageCPP/      ← racine du dépôt GitHub
├── .git/               ← géré par Git — NE JAMAIS TOUCHER
├── .github/
│   └── workflows/
│       └── doxygen.yml ← workflow CI/CD unique pour tous les projets
├── .gitignore          ← doit exclure tous les docs/ générés
├── AdresseIP/
│   ├── ipv4.h
│   ├── ipv4.cpp
│   ├── main.cpp
│   ├── Doxyfile        ← chemins RELATIFS obligatoires
│   └── docs/           ← généré, ignoré par Git
│       └── html/
├── Compteur2/
│   ├── Compteur.h
│   ├── Compteur.cpp
│   ├── Doxyfile
│   └── docs/
└── FormeGeometrique/
    ├── ...
    ├── Doxyfile
    └── docs/
```

12.2. Préparer le Doxyfile de chaque projet

Avant tout commit, vérifiez que ces deux paramètres sont en chemins **relatifs** :

```
# Vérifier depuis la racine du dépôt
grep "^INPUT" MonProjet/Doxyfile      # → INPUT = .
grep "^OUTPUT_DIRECTORY" MonProjet/Doxyfile # → OUTPUT_DIRECTORY = docs
```

Si les chemins sont absolus, corrigez-les avec sed :

```
cd MonProjet
sed -i 's|^INPUT\s*=.*|INPUT = .|' Doxyfile
sed -i 's|^OUTPUT_DIRECTORY\s*=.*|OUTPUT_DIRECTORY = docs|' Doxyfile

# Vérification
grep "^INPUT" Doxyfile      # → INPUT = .
grep "^OUTPUT_DIRECTORY" Doxyfile # → OUTPUT_DIRECTORY = docs
```

Piège fréquent — chemins absolus dans le Doxyfile

Doxywizard génère souvent des chemins absolus vers votre machine locale :

```
INPUT = /home/USERS/PROFS/pcruchet/ApprentissageCPP/AdresseIP/
```

Ces chemins n'existent pas sur le runner GitHub → le workflow échoue.

Il faut impérativement utiliser des chemins relatifs (voir 12.2).

12.3. Configurer le .gitignore

Le contenu de docs/ est généré automatiquement par le workflow — il ne doit pas être versionné dans main.

```
# Depuis la racine du dépôt, ajouter une ligne par projet
echo "AdresseIP/docs/" >> .gitignore
echo "Compteur2/docs/" >> .gitignore
echo "FormeGeometrique/docs/" >> .gitignore

# Si des docs sont déjà committées, les retirer du suivi Git
git rm -r --cached AdresseIP/docs/
git rm -r --cached Compteur2/docs/

git add .gitignore
git commit -m "chore: ignorer les docs générées"
git push origin main
```

12.4. Créer le dossier .github/workflows/

Le dossier .github/ est **distinct** de .git/. Vous le créez vous-même — GitHub le reconnaît et exécute les fichiers qu'il contient.

Dossier	Créé par	Rôle
.git/	Git automatiquement	Base de données interne de Git — ne jamais modifier
.github/	Vous	Instructions pour GitHub (CI/CD, templates, workflows)

```
# Créer le dossier depuis la racine du dépôt
mkdir -p .github/workflows

# Créer le fichier workflow
nano .github/workflows/doxygen.yml
# (coller le contenu de la section 12.5, puis Ctrl+O / Ctrl+X)
```

12.5. Fichier doxygen.yml — workflow complet

Ce workflow gère **tous les projets** du dépôt en un seul fichier. Chaque projet est un job indépendant. clean: false est obligatoire pour que les projets ne s'effacent pas mutuellement sur la branche gh-pages.

```
name: Documentation Doxygen

on:
  push:
    branches:
      - main
  workflow_dispatch:      # Lancement manuel depuis GitHub > Actions

jobs:
  # — Projet AdresseIP —————
  deploy-AdresseIP:
    runs-on: ubuntu-latest
    permissions:
      contents: write      # Nécessaire pour pousser sur gh-pages
    steps:
      - name: Checkout du code
        uses: actions/checkout@v4
```

```

- name: Installer Doxygen et Graphviz
  run: |
    sudo apt-get update -qq
    sudo apt-get install -y doxygen graphviz

- name: Générer la documentation
  working-directory: AdresseIP
  run: doxygen Doxyfile

- name: Créer .nojekyll
  run: touch AdresseIP/docs/html/.nojekyll

- name: Déployer sur GitHub Pages
  uses: JamesIves/github-pages-deploy-action@v4
  with:
    branch: gh-pages
    folder: AdresseIP/docs/html
    target-folder: AdresseIP      # sous-dossier sur gh-pages
    clean: false                  # NE PAS effacer les autres projets

# — Projet Compteur2 —————
deploy-Compteur2:
  runs-on: ubuntu-latest
  permissions:
    contents: write
  steps:
    - uses: actions/checkout@v4
    - run: sudo apt-get install -y doxygen graphviz
    - working-directory: Compteur2
      run: doxygen Doxyfile
    - run: touch Compteur2/docs/html/.nojekyll
    - uses: JamesIves/github-pages-deploy-action@v4
      with:
        branch: gh-pages
        folder: Compteur2/docs/html
        target-folder: Compteur2
        clean: false

# — Ajouter un job par nouveau projet —————
# deploy-FormeGeometrique:
#   ...même structure, adapter les noms...

```

12.6. Premier déploiement — procédure complète

Séquence à suivre la première fois pour un nouveau projet :

```

# 1. Vérifier les chemins relatifs dans le Doxyfile
grep "^INPUT" MonProjet/Doxyfile      # → INPUT = .
grep "^OUTPUT_DIRECTORY" MonProjet/Doxyfile # → OUTPUT_DIRECTORY = docs

# 2. Ajouter le dossier docs/ au .gitignore
echo "MonProjet/docs/" >> .gitignore

# 3. Si docs/ est déjà suivi par Git, le retirer
git rm -r --cached MonProjet/docs/    # (ignoré si docs/ n'existe pas encore)

```

```
# 4. Ajouter le job dans .github/workflows/doxygen.yml
# (copier le bloc deploy-Compteur2 et adapter les noms)

# 5. Vérifier que les permissions sont en "Read and write"
# GitHub → Settings → Actions → General → Workflow permissions
# ● Read and write permissions

# 6. Committer et pousser
git add .gitignore MonProjet/Doxyfile .github/workflows/doxygen.yml
git commit -m "ci: ajout documentation MonProjet"
git push origin main

# 7. Vérifier dans GitHub → Actions → doxygen.yml → 
```

12.7. URLs d'accès

Une fois le workflow vert et GitHub Pages configuré :

```
# Documentation AdresseIP
https://pcruchet.github.io/ApprentissageCPP/AdresseIP/

# Documentation Compteur2
https://pcruchet.github.io/ApprentissageCPP/Compteur2/

# Documentation FormeGeometrique
https://pcruchet.github.io/ApprentissageCPP/FormeGeometrique/
```

12.8. Récapitulatif des avantages

Aspect	Bénéfice
Régénération auto	Chaque git push sur main régénère et redéploie la doc en ~30s
Pas de HTML dans main	L'historique git reste propre — seul le code source est versionné
Branche gh-pages auto	Créée automatiquement au premier run réussi, jamais à créer à la main
Multi-projets	Un seul fichier doxygen.yml gère tous les projets du dépôt
Chemins relatifs	Le Doxyfile fonctionne en local ET sur le runner GitHub
Permissions	Read and write à activer une fois dans Settings → Actions

13. Checklist — documentation complète

13.1. Avant de générer

- Chaque fichier .h et .cpp a un en-tête @file / @author / @date / @brief
- Chaque classe a un bloc @class avec @brief et @details
- Chaque attribut a un commentaire ///
ou /** */
- Chaque méthode publique a @brief, @param (un par paramètre), @return et @throws si applicable
- Les groupes @defgroup sont définis si le projet comporte plusieurs modules
- Le Doxyfile a EXTRACT_ALL = YES, HAVE_DOT = YES, UML_LOOK = YES
- Le fichier .nojekyll est présent dans docs/html/

13.2. Après génération

- Ouvrir docs/html/index.html et vérifier le panneau de navigation latéral
- Vérifier les diagrammes de classes (onglet "Classes" > "Class Hierarchy")
- Vérifier les call graphs sur au moins une méthode
- Vérifier la barre de recherche (nécessite un serveur HTTP ou SEARCHENGINE = YES)
- Committer et pousser → vérifier l'URL GitHub Pages → La documentation est mise à jour systématiquement grâce au workflow.

13.3. Pour le déploiement GitHub Actions

- Le dossier .github/workflows/doxygen.yml est créé et commité
- Le dossier docs/ est listé dans .gitignore (créé automatiquement par le workflow)
- Les permissions sont en "Read and write" dans Settings → Actions → General
- GitHub Pages est configuré sur "gh-pages / (root)" ou "GitHub Actions" (section 11.4)
- Le workflow est vert (✅) dans l'onglet Actions après le premier push
- L'URL GitHub Pages affiche bien la documentation

Ressources utiles

- Documentation officielle Doxygen : <https://www.doxygen.nl/manual/>
- Liste complète des balises : <https://www.doxygen.nl/manual/commands.html>
- Graphviz : <https://graphviz.org/>
- JamesIves deploy action : <https://github.com/JamesIves/github-pages-deploy-action>
- GitHub Pages documentation : <https://docs.github.com/en/pages>